mathematics

MDPI

Article

A Scalable Post-Processing Pipeline for Large-Scale Free-Space Multi-Agent Path Planning with PIBT

Arjo Chakravarty [1,2,*], Michael X. Grey [1], M. A. Viraj J. Muthugala [2] and Rajesh Mohan Elara [2]

[1] Intrinsic LLC, 438A Alexandra Road, #04-11/12 Block A, Alexandra Technopark, Singapore 119967, Singapore
[2] Engineering Product Development Pillar, Singapore University of Technology and Design, 8 Somapah Road, Singapore 487372, Singapore; viraj_jagathpriya@sutd.edu.sg (M.A.V.J.M.); rajeshelara@sutd.edu.sg (R.M.E.)
* Correspondence: arjo_chakravarty@mymail.sutd.edu.sg

Abstract

Free-space multi-agent path planning remains challenging at large scales. Most existing methods either offer optimality guarantees but do not scale beyond a few dozen agents or rely on grid-world assumptions that do not generalize well to continuous space. In this paper, we propose a hybrid, rule-based planning framework that combines Priority Inheritance with Backtracking (PIBT) with a novel safety-aware path smoothing method. Our approach extends PiBT to eight-connected grids and selectively applies string-pulling-based smoothing while preserving collision safety through local interaction awareness and a fallback collision resolution step based on Safe Interval Path Planning (SIPP). This design allows us to reduce overall path lengths while maintaining real-time performance. We demonstrate that our method can scale to over 500 agents in large free-space environments, outperforming existing any-angle and optimal methods in terms of runtime, while producing near-optimal trajectories in sparse domains. Our results suggest this framework is a promising building block for scalable, real-time multi-agent navigation in robotics systems operating beyond grid constraints.

Keywords: multi-agent systems; path planning; path smoothing; multi-agent pathfinding

MSC: 68R01

1. Introduction

This paper explores the idea of a practical free-space multi-agent planner that scales to 100 s of robots. We build on Priority Inheritance BackTracking (PIBT) [1], which is currently one of the leading rules-based configuration generators for moving multiple agents. Thus far PIBT has largely been used in traditional grid-like multi-agent path-finding (MAPF) settings. Here we ask if it is possible to extend PIBT to free space planning. We combine path smoothing, and safe-interval path planning (SIPP) with PIBT to introduce a new rules-based planner. While we do not provide any optimality guarantees, we show that the planner is able to find solutions significantly faster than any-angle continuous-time conflict-based search (AA-CCBS) [2], which was the first any-angle planner. To do so, we break down this work into two pieces: First we simply extend PIBT from a four-connected domain to an eight-connected domain. We empirically show that this reduces path lengths. Then we apply a multi-agent safety-aware string-pulling mechanism to smooth the paths. This approach has similarities to navigation frameworks like Nav2 [3], which do not always need to use optimal any-angle planning. Our key contributions are:

Check for updates

Academic Editor: Jüri Majak

Received: 2 March 2026
Revised: 18 March 2026
Accepted: 21 March 2026
Published: 3 April 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

- **Algorithm Mechanism:** We propose a hybrid framework that bridges the gap between reactive rule-based configuration generation (PIBT) and post hoc trajectory optimization. This is achieved by extending PIBT to eight-connected domains and introducing a novel "Safety-Aware" smoother that preserves the local coordination logic of PIBT (Section 3).
- **Safety-Aware Smoothing:** We introduce a heuristic that identifies "critical interaction regions" where smoothing is likely to introduce collisions. By selectively inhibiting smoothing in these regions, we significantly reduce the need for computationally expensive collision repair steps (Section 3.2), as validated in our ablation study.
- **Scalability vs. Any-Angle Optimality:** We provide an empirical comparison against AA-CCBS+SIPP, the state-of-the-art method for any-angle optimal planning. We demonstrate that our approach achieves near-optimal path lengths (within 0.8% of optimal on average path length) while maintaining near-linear $O(n)$ scalability, enabling planning for up to 500 agents in scenarios where optimal solvers fail to find solutions (Section 4.1).

2. Related Work

2.1. Multi-Agent Path Finding (MAPF)

Multi-agent path finding (MAPF) is a vibrant domain. A lot of work has focused on solving MAPF optimally. Specifically, the community has targeted grid-like settings as these are good enough approximations of the real world warehouse settings and a good starting point for research. Currently there are several families of MAPF algorithms that can be broadly broken down into the following:

- Conflict-Based Search (CBS) [4]: These algorithms rely on a two-level search. The low-level search runs traditional path-finding algorithms. When a conflict arises, the high level search creates child nodes in a tree that split on the specific conflict. This approach has been shown to be very effective in the 10–100-robot setting [5]. Additionally, its simplicity has lent itself to many extensions [5,6]. However, it often suffers from exploring too many conflicts and may have unpredictable runtimes [7,8].
- Compilation-Based: Compilation-based methods decompose the MAPF problem to equivalent NP-hard problems such as MILP [9] or SAT [10]. This approach has limited scalability and is hard to extend [9].
- Rules-Based: This category of solvers relies on bespoke hand-engineered heuristics. A recent breakthrough by Keisuke et al. [1] introduced PIBT as a viable rule-based approach to multi-agent solving with practical scalability.
- Priority-Based: These are the simplest solvers and scale well; however, they often run into deadlocks [1].
- Learning-Based: Learning-based approaches such as PRIMAL [11], SCRIMP [12] and MAPF-GPT [13] are growing in popularity in part due to the recent advances in Deep Reinforcement Learning (RL) and improved GPU performance. Most learning-based planners need to be bootstrapped via imitation learning from other types of planners, and then they refine their strategies via RL. They are useful for decentralized, partially observable local behaviors [11–13].

Aside from these broad categories, there are many varieties of planners that explore hybrid strategies. We chose to work with PIBT as it can be used as a component by other planners and provides unprecedented scalability. Grid-world representations have been a central focus of MAPF research. Several standard benchmarks exist for these [14,15]. These provide an excellent but simple base for researching new search schemes. However, many real-world robotics scenarios would benefit from not being constrained to the restrictive four-connected grid approach. Adreychuk et al. [2] proposed AA-CCBS, which

builds on CCBS and is combined with SIPP to achieve optimal any-angle planning. A similar approach was attempted by Open-RMF's MAPF library as well [16]. While these provide optimality guarantees, many applications prefer a solution to be available (even if suboptimal) than having to wait for a solver to arrive at the optimal solution.

PIBT was proposed by Keisuke et al. as a "Configuration Generator" for MAPF solvers. It provides a certain simple set of rules that gives feasible configurations quickly. PIBT combines the notion of priority-based planning together with deadlock resolution methods. Specifically, PIBT uses priority inheritance to prevent priority inversion and backtracking to resolve deadlocks. It has been proven to be complete for the multi-agent pickup and delivery (MAPD) problem on bi-connected graphs [1]. Its speed makes it practical for many modern MAPF planners to use as a low-level planner [8,17].

2.2. Path Smoothing and Single-Agent Navigation

When dealing with free-space navigation, theta* is commonly used for any-angle path finding [18]. However, theta* can be expensive as it needs to perform line-of-sight checks [19]. Several approaches to solve this have been proposed, including lazy-theta* [19]. For our purpose, it was more important to be able to scale to more robots than to provide perfect optimality. This is important for large-scale applications as optimal MAPF is NP-hard. Suboptimal MAPF is believed to be solvable in poly-time. In fact, PIBT has $O(|A|(\Delta(G) + logA))$ time complexity [1] per times tep. In conventional navigation, like Nav2, it is a common practice to use path planners that do not guarantee any-angle optimality and then smooth the paths [3] post hoc. Some common path smoothers include Savitzky–Golay, string pulling [20] and constraint path smoothing [3]. However, while these methods are effective for single-agent navigation where obstacles are largely static, they are often "safety-blind" in multi-agent contexts. The challenge with post hoc smoothing in MAPF is that it frequently introduces unintended side effects: a "straight-line" optimization that is safe in isolation may result in spatiotemporal collisions when other agents simultaneously adjust their own trajectories [21]. Traditional solutions to this problem involve either global re-optimization, which is computationally expensive and poorly suited for hundreds of agents, or relying on local reactive avoidance (like ORCA [22]), which can lead to oscillations or deadlocks in dense areas. We propose a procedure that significantly reduces the number of intersections by proactively identifying and preserving critical interaction segments, thus bridging the gap between efficient post hoc smoothing and multi-agent safety.

2.3. Eight-Connected Grid—A Good Enough Approximation

There are many ways to represent a free space. In the domain of single agent planning, it is common to use a grid. When planning for one agent, visibility graphs or theta* can accelerate optimal path finding. However, the improvement that theta* has over classic eight-connected grids is roughly 8% [23].

3. Proposed Algorithm

We propose a path smoother that guarantees collision-free paths while reusing the results of eight-connected PIBT.

3.1. Eight-Connected PIBT

There are two possible ways to extend PIBT to the 8-connected case: The first is to simply extend the "get neighbors" function and update the collision check to take into account the diagonal cases. This is shown in Figure 1. The second more complicated way would be to extend priority inheritance over multiple cells. For this work we only consider the first approach.

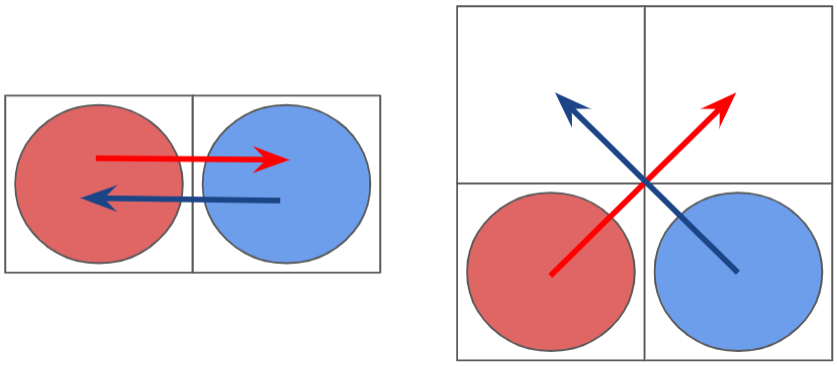


Figure 1. On the (**left**) we have the traditional swap conflict that is handled by classic PIBT. On the (**right**) we have additional swap conflicts that needs to be handled in 8-connect PIBT.

Despite a speed penalty for the higher branching factor, we find 8-connected PIBT is able to scale to 100 s of robots. Additionally, we find paths proposed by 8-connected PIBT to be shorter than their 4-connected counter parts. One of the pitfalls of 8-connected PIBT however is that the travel time in the diagonal is not the same as the travel time in the cardinal directions. For the purpose of this paper, we assume the robots can operate at a range of speeds, with the maximum speed being more than the time it takes to cover a diagonal of a single cell. It can be shown that any line that is contained in a square has to be shorter than or equal to the length of its diagonal; thus, we can use this as the max robot speed. Using 8-connected PIBT is important as, when we perform string pulling, the newly formed line that shortens the path can deviate significantly from the original path. With the 4-connected variant, this can be problematic as we may aggressively pull strings, leading to more collisions (see Section 4).

3.2. Safety-Aware String Pulling for Path Smoothing

The key insight our algorithm uses is that PIBT negotiations are local; hence, we hand control to PIBT when agents are *directly interacting* and hand control to our smoother when agents are not directly interacting, as illustrated in Figure 2.

String pulling is a classic technique for shortening paths produced by A* [20]. However, when dealing with free-space planning and the output of PIBT, it is important to take into account how agents interact. In this section we directly borrow terminology from the original PIBT paper [1]. The interaction of agents is governed by the notion of *interacting agents*. *Interacting agents* are defined by the fact that PIBT is a local-first configuration generator. If two agents are located within two hops of each other, they are considered to be *directly interacting*. This two-hop threshold is chosen because the backtracking mechanism in PIBT typically resolves conflicts within a two-cell radius in an 8-connected grid; beyond this range, the local configuration generation of one agent is decoupled from the other. A group of *interacting agents* is defined transitively over chains of directly interacting agents. It follows that every *interacting agent* must be *directly interacting* with some agent. Therefore, a simple space–time search should reveal the neighbors of an agent. If there are interactions, we do not attempt to smooth them. This alone does not guarantee the safety of the newly smoothed path. We must ensure that our pulled string does not cross into regions of *critical interactions*. We define an area of *critical interaction* as an area that is within 1 hop of another agentm, as the agent may enter this region in the next time step and was not expecting an obstacle in the original plan (see Figure 3). This one-hop "buffer" is a conservative empirical choice that aligns with the maximum displacement of an agent in a single time step, ensuring that the smoothed path does not intersect the reachable set of any neighboring agent during the next control cycle.

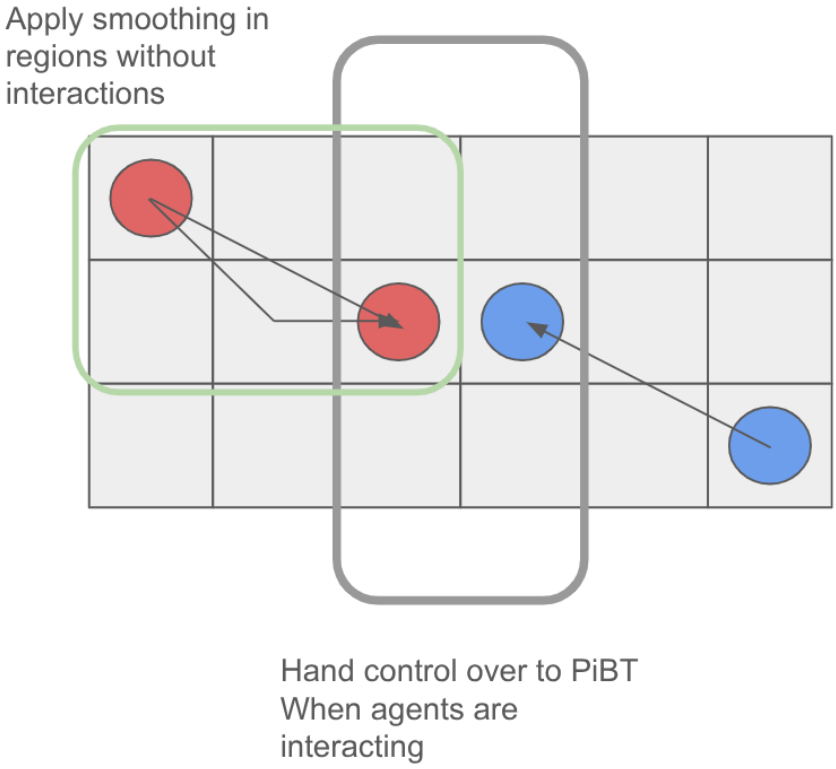


Figure 2. Our key insight is that since the interactions in PIBT are local, path smoothing should happen only when agents are far from each other.

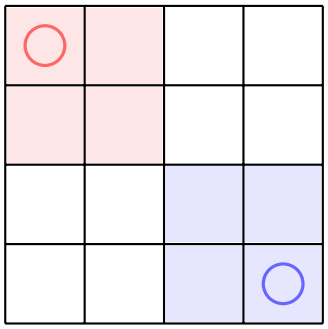


Figure 3. Areas of critical interactions: In this case the blue agent can move into any of the blue colored squares and the same for the red agent. Other agents should avoid pulling into these regions.

We refer our reader to Algorithm 1. If an agent is an *interacting agent*, we make sure the agent reaches the critical interaction at the allotted time, and we do not perform any smoothing while agents are in critical interactions (see line 31 in Algorithm 1). Our line-of-sight check during string pulling takes into account regions of *critical interaction* by treating such regions as a time-bound obstacle (line 9).

Even after these restrictions, we do not have a guaranteed collision-free path. It is possible that 2 newly pulled strings might intersect at the same time and place. To avoid this, we rely on a complementary solver. In this case, we implement SIPP with a priority-based scheme to prevent the two agents from colliding. In our benchmarks we found that the number of intersections were low, and all of them were possible to resolve using SIPP. However, a more complete solution would be an interesting direction for future work.

Algorithm 1 PIBT path smoothing.

Parameters: PIBT path smoothing
Output: collision-free smoothed path $a^{1:N}$

procedure STRING-PULL(T, O, $C^{1:N}(t)$)
    T: the trajectory consisting of tuples (t_i, x_i, y_i)
    O: the list of obstacles
    C: a list of regions of critical interaction
    $T_{new} = \{T(0)\}$
    for $t_i < t_{MAX}$ **do**
        for $t_j \in [t_{i+1} \dots t_{MAX})$ **do**
            $t_{next} \leftarrow t_j$
            if $\neg$line_of_sight$(T[t_i], T[t_j], O, C)$ **then**
                break
            end if
        end for
        $T_i \leftarrow$ LINEARINTERPOLATION($T[t_i]$, $T[t_{next}]$)
        APPEND(T_{new}, T_i)
        $t_i \leftarrow t_{next}$
    end for
    return T_{new}
end procedure
procedure PIBT-SMOOTHER($s^{1:N}(t)$, O)
    $s^{1:N}(t)$: agent trajectories from PIBT solver
    O: Llst of obstacles
    $C^{1:N}(t) \leftarrow s^{1:N}(0)$
    for time $t \in 0 \dots t_{MAX}$ **do**
        for agent $n \in 1 \dots N$ **do**
            if CRITICALINTERACTION($s^n(t)$) **then**
                APPEND($C^n(t)$, $s^n(t)$)
            end if
        end for
    end for
    $C^{1:N}(t_{MAX}) \leftarrow s^{1:N}(t_{MAX})$
    for agent $n \in 1 \dots N$ **do**
        for $s_0, s_1 \in$ CONSECUTIVEPAIRS(C^n) **do**
            $T^n \leftarrow$ TRAJECTORYSEGMENT(s_0, s_1)
            $T^n \leftarrow$ STRINGPULL(T^n, O, $C^{1:N}$)
        end for
    end for
    for agent $n_1 \in 1 \dots N$ **do**
        for agent $n_2 \in n_1 + 1 \dots N$ **do**
            if INTERSECTION(T^{n_1}, T^{n_2}) **then**
                APPLYSIPP(T^{n_1}, T^{n_2})
            end if
        end for
    end for
    return
end procedure

4. Benchmarks

4.1. Evaluation Criteria

To rigorously evaluate the performance and scalability of our proposed framework, we define the following metrics:

- **Near-optimality:** We quantify path quality by comparing the total arclength of the generated trajectories to the optimal any-angle solutions provided by AA-CCBS+SIPP,

where available. "Near-optimal" is defined as a total solution cost within a 1% margin of the optimal baseline.

- **Makespan and temporal consistency:** We monitor the makespan (the total time to reach the final goal) to ensure that the post-processing smoothing does not violate the temporal constraints established by the PIBT configuration generator. By design, our smoother preserves the original arrival times at critical interaction segments to maintain the safety guarantees of the PIBT plan.
- **Computational efficiency:** this is measured by the wall-clock time required to generate the initial PIBT plan and the subsequent smoothing and collision repair steps.

4.2. Experimental Setup

We made available a Python 3.14 implementation of our algorithm (https://github.com/arjo129/pypibt (accessed on 20 March 2026). We benchmarked this against CCBS+SIPP as implemented in Open-RMF's MAPF library [16]. Our benchmarks use the MAPF benchmarks by Stern et al. [15]. In our results table, we reuse the same scenario names as those in [15]. We benchmarked the post-smoothing behavior against a pre-smoothing behavior and show our improvements. We also show the cost reduction as compared to CCBS+SIPP. All benchmarks were run on a Thinkpad P50 with a 16-core i7-11850H processor and 64 GB RAM. We provide the raw benchmark results for 800 scenarios in the MAPF benchmark in this url (https://github.com/arjo129/pypibt/tree/main/docs/results (accessed on 20 March 2026). For the purpose of brevity in our discussion, we will highlight the two maps in which we see the best performance (Berlin_256_256) and worst performance (Warehouse-20-4-10-2-1) for all our tables. These maps are reproduced for the reader in Figure 4. They represent two extremes: the Warehouse-20-4-10-2-1 instance is extremely grid-like, where the shortest paths are likely to follow the cardinal directions, whereas the Berlin_256_256 instance has lots of free space, enabling a large amount of optimization to take place.

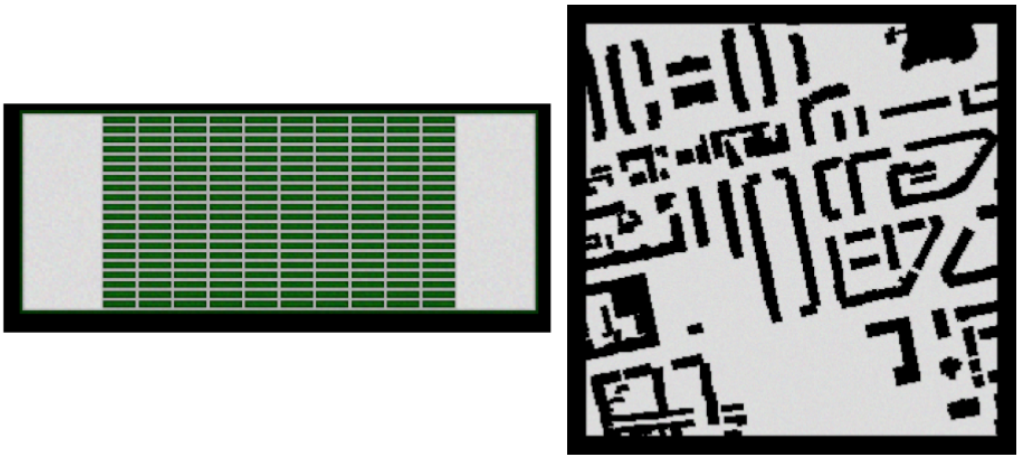


Figure 4. The two maps we refer to most frequently in our paper. We ran experiments on all the other maps in the benchmarks but frequently refer to these. On the (**left**) is Warehouse-20-4-10-2-1; on the (**right**) is Berlin_256_256. The names we use are derived from the original benchmark [15].

Table 1 compares the solution times for a set of maps from the MAPF benchmark. As we can see CCBS+SIPP can scale to four robots easily but struggles when we reach the 10 robots range. On the other hand, we find that the PIBT+path-smoothing approach is able to scale to the 100-robot range.

Comparing AA-CCBS+SIPP with our path smoother presents inherent discrepancies, primarily because AA-CCBS+SIPP includes optimality guarantees, which our approach does not offer. To quantify this gap, we compare the total arclength of the generated paths in

Table 2. We find that PIBT+path smoothing produces paths that are, on average, 0.8% longer than the optimal baseline for sparse set of agents across all maps in the MAPF dataset. It is important to clarify the statistical context of this figure: since optimal any-angle planners like AA-CCBS+SIPP fail to scale beyond a limited number of agents (typically ≤ 15), this average is computed exclusively over the subset of benchmark instances where the baseline solver successfully converged. This represents 1906 out of 4948 scenarios. While this comparison does not capture the behavior in ultra-high-density scenarios (where no optimal baseline is available), Table 2 provides a representative view of the performance hit incurred by our rule-based approach in solvable domains.

Table 1. Solution times compared to CCBS+SIPP.

Scenario Name	Number of Agents	Solve Time CCBS+SIPP	Solve Time PIBT+PS
Warehouse-20-40-10-2-1 random-3	2	0.249 s	0.165 s
	4	0.595 s	0.547 s
	10	2.15 s	1.55 s
	100	Unsolved	13.5 s
Berlin_1_256 random-20	2	0.28 s	0.29 s
	4	Unsolved	0.67 s
	100	Unsolved	27.8 s

Table 2. Arclength compared to CCBS+SIPP.

Map Name	Number of Agents	Arclength CCBS+SIPP	Arclength PIBT+PS
warehouse-20-40-10-2-1 random-3	2	256.4	256.4
	4	730.4	735.1
	10	1945.0	1960.6
	15	2591.9	2625.3
Berlin_1_256 random-20	2	175	178
	4	Unsolved	440
	100	Unsolved	16,589.3

Table 3 shows the impact of our path smoothing in comparison to vanilla PIBT. The gap between vanilla PIBT and our path smoothing increases with the number of agents. This is shown even more clearly when dealing with large numbers of agents in Figure 5.

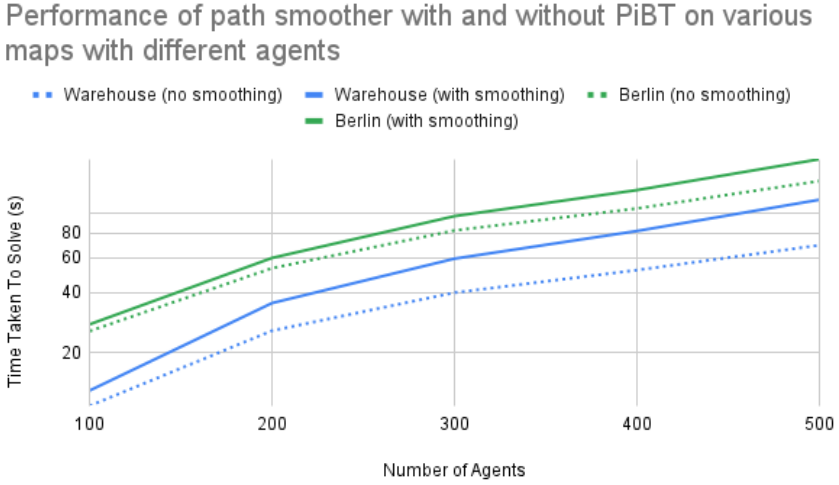


Figure 5. Performance of smoother vs. without smoother on select maps in the 100 s of robots range.

Essentially, for small numbers of agents, the overhead of computing path smoothing is low; however, as the numbers increase, the overhead begins to consume a significant chunk of time.

Table 4 is an ablation study on taking away certain components and seeing how frequently we need to fall back to using some local collision resolution strategy. The first configuration we compare using four-connected PIBT vs. eight-connected PIBT for the number of collisions. Additionally, we compare the "Safety-Aware" version to a vanilla string pulling, which ignores the *critical regions*. We find that the *critical-region*-aware versions produce an order-of-magnitude fewer intersections, allowing us to minimize the number of times we need to call our fallback planner.

Table 3. Solution times with and without path smoothing.

Map Name	Number of Agents	8-Connect PIBT	8-Connect PIBT Path Smoothing
Warehouse-20-40-10-2-1 random-3	2	0.15 s	0.17 s
	4	0.50 s	0.55 s
	10	1.4 s	1.5 s
	15	1.9 s	2.1 s
	20	2.2 s	2.5 s
	100	11.2 s	13.5 s
Berlin_1_256 random-20	2	0.28 s	0.30 s
	4	0.63 s	0.67 s
	10	2.5 s	2.6 s
	15	3.9 s	4.1 s
	20	5.2 s	5.4 s
	100	25.7 s	27.8 s

Table 4. Intersections introduced by path smoothing.

Map Name	Number of Agents	4-Connect PIBT+PS	8-Connect PIBT+PS	Ignoring Critical Regions
Berlin_1_256	50	3	0	13
Berlin_1_256	100	6	2	63
Berlin_1_256	200	12	11	170
Berlin_1_256	500	25	21	980

Finally, we also benchmark the effectiveness of the path smoothing by showing the total path length reduction. Table 5 shows the path length reduction in the best-case scenario. For four robots we can reduce the path length by as much as 27.8%. The general trend is the greater the density of robots, the less effective our path-smoothing approach. If we control for *critical interactions*, we find that the path reduction is around 16.0%, except for cases where the robot density is very low.

Table 5. Path length reduction vs. number of agents.

Map Name	Number of Agents	Total Path Length Reduction	Normalized Path Length Reduction
Berlin_1_256	4	27.8%	27.8%
Berlin_1_256	10	15.0%	16.0%
Berlin_1_256	15	14.3%	16.2%
Berlin_1_256	20	12.9%	16.7%
Berlin_1_256	100	10.0%	15.8%

5. Discussion

Figures 6 and 7 show the aggregate effects of our path-smoothing scheme on a large-scale scenario. As seen in Figure 7, the trajectories are significantly less jagged than the unsmoothed baselines in Figure 6, with a close-up comparison provided in Figure 8.

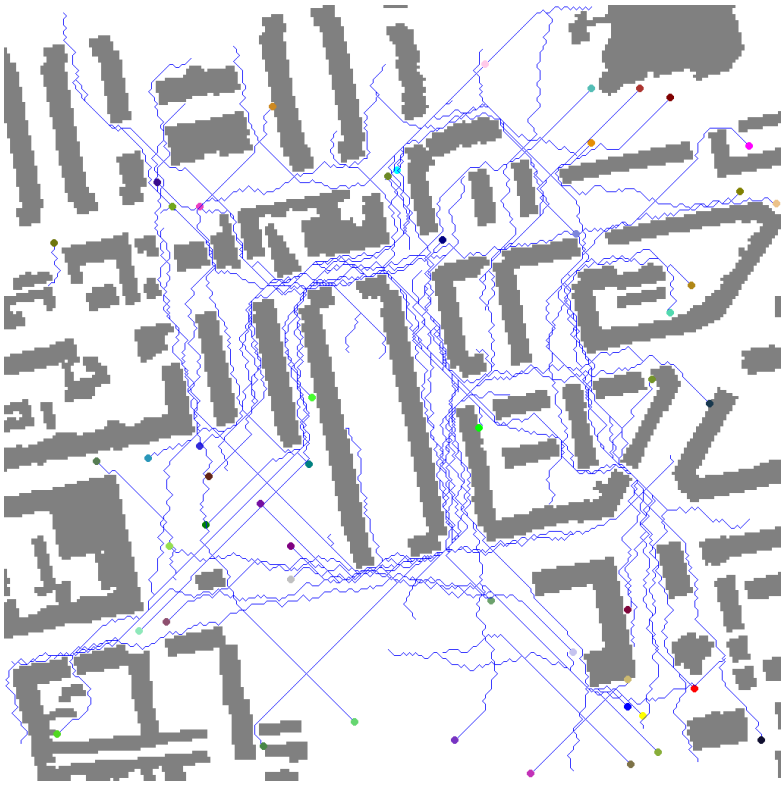


Figure 6. Eight-connected PIBT without path smoothing.

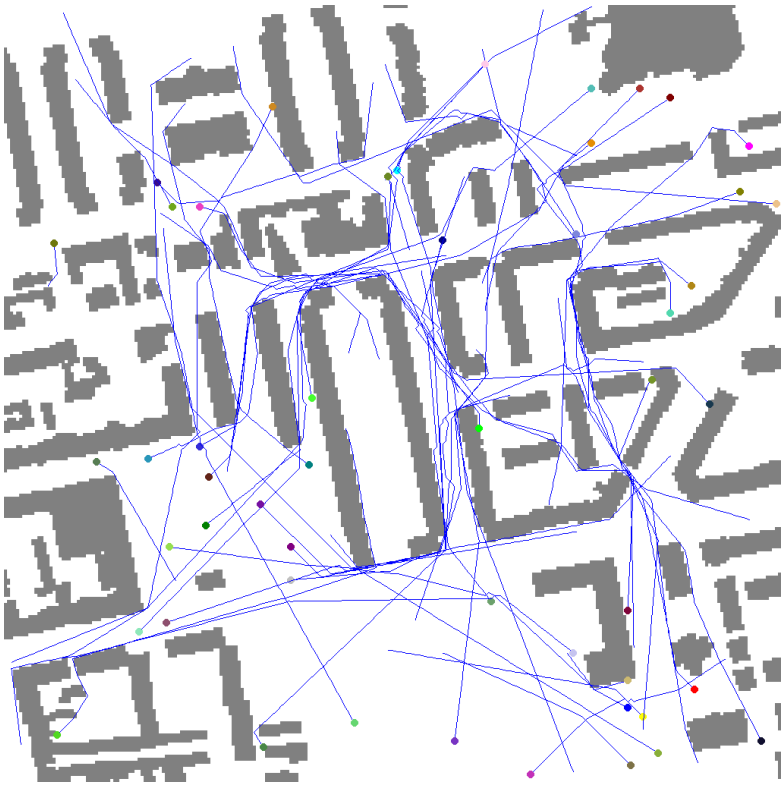


Figure 7. Eight-connected PIBT with string path smoothing. Notice the improved paths compared to Figure 6.

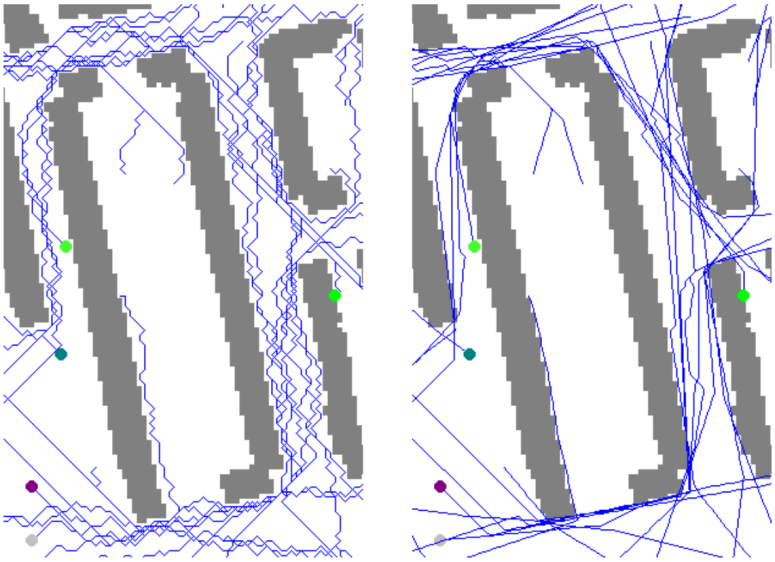


Figure 8. A magnified view comparing the smoothing action in Figures 6 and 7. The (**left**) side shows the unsmoothed version, and the (**right**) shows the smoothed version.

To better visualize individual agent behaviors and the specific mechanism of our safety-aware smoother, we provide a qualitative comparison with per-agent color coding for 50 agents in Figure 9. This comparison highlights how naïve string pulling (middle panel) often results in spatiotemporal collisions (marked in red) in dense scenarios. In contrast, our safety-aware approach (right panel) identifies critical interaction regions and selectively inhibits smoothing, thereby preserving the safety of the original PIBT plan (left panel) while still achieving significant path shortening. This visual evidence complements the ablation study in Table 4, which shows that ignoring these critical regions leads to an order-of-magnitude increase in new collisions.

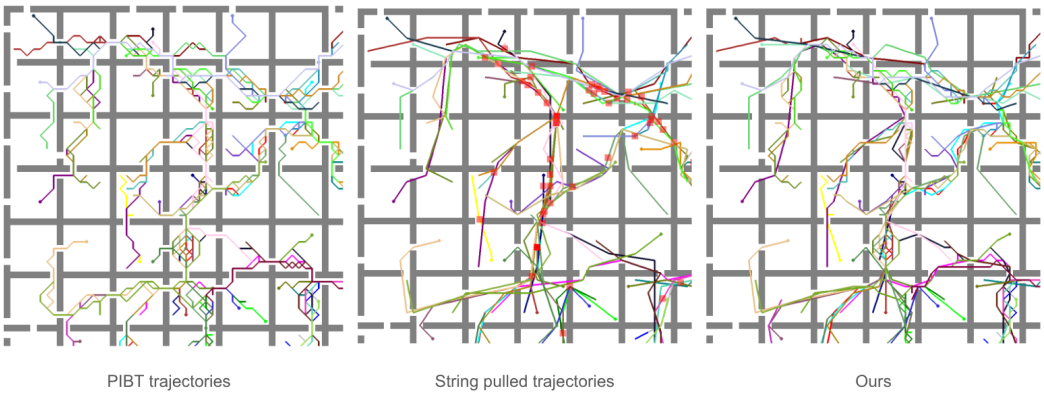


Figure 9. Comparison of smoothing strategies on a 50-agent scenario. Each agent and its path has been individually colored. Red squares indicate temporal–spatial collisions where agents occupy the same space. (**Left**): original PIBT plan on an 8-connected grid. (**Middle**): naïve string-pulling smoothing; (**Right**): proposed safety-aware smoothing, where smoothing is inhibited within critical interaction regions, preserving safety while reducing path length.

Compared to AA-CCBS+SIPP, we show that our PIBT-based approach has the ability to scale more effectively. As seen in Table 1, our solver can find solutions for 10 s–100 s of agents in a practical timeframe, whereas AA-CCBS+SIPP can achieve free-space planning for 2–15 agents. It is important to note that AA-CBS+SIPP offers optimality guarantees, which we do not. However as the optimal variant of MAPF is known to be NP-hard [24], in many real-world scenarios, we would prefer methods with a bounded runtime.

The path-smoothing approach is particularly effective when there are large empty areas with geometries that may not fit well into a grid world. This explains the extreme effectiveness in the Berlin map, where path smoothing was able to reduce path length by up to 27.8% of the original path length proposed by PIBT. Conversely, path smoothing has almost no effect in worlds with narrow grid patterns like the warehouse world, where the improvements via path smoothing are as low as 0%.

The path-smoothing approach is also not appropriate for reducing the makespan as the makespan of the trajectories is preserved whenever there is any *critical interaction*. A consequence of this is shown in Table 5, where the total path length reduction decreases as agent density increases. This is a mechanical consequence of our safety-aware heuristic: as agents become more densely packed, larger proportions of their paths are classified as critical interaction regions, reducing the "smoothable" fraction of the total trajectory. To isolate the smoother's intrinsic performance from this density effect, we report the "Normalized Path Length Reduction" in Table 5, which is computed only over the segments that were actually subjected to smoothing. This normalized metric demonstrates that the smoothing mechanism remains highly effective on available free-space segments regardless of overall agent density, confirming that the declining total gain is a deliberate trade-off for safety rather than a degradation of the smoother's capabilities.

Perhaps the most significant drawback of our algorithm is the need to check for intersections and resolve them via traditional MAPF techniques at the end of our smoothing if new trajectory intersections arise. Our experiments show that such intersections do arise when handling hundreds of robots; however, the number of new intersections stays well below 20 in most cases. Such intersections usually arise for a limited number of robots, so their resolution is relatively easy to compute, and the solutions for each intersection can be computed independent from each other.

Based on our findings in Table 4, we see that considering areas of *critical interaction* is incredibly important for avoiding the need to fall back to local collision avoidance techniques.

Apart from the specific cases highlighted in our discussion, we observe an average path length reduction of 7.3% across all tested scenarios and had an average overhead of 19% for larger instances ($\geq$100 robots). The overhead was much smaller in smaller instances, where it was as low as 2%.

6. Conclusions and Future Work

We showed a method for path smoothing over PIBT. Our key insight was that we used the local properties of PIBT to reduce the number of collisions introduced when smoothing the path. By partitioning the path into segments where PIBT has full control due to high robot density and segments that are safe to smooth, we showed that we reduced the path length in maps with large amounts of free space while introducing a minimal number of new trajectory intersections. This approach allowed us to approximate free space traversal of robots at a much larger scale than conventional AA-CCBS+SIPP.

Several factors that may be relevant to real-world applications are not modeled in our method. First, we did not look at the Kino-dynamics of the robot. This can be problematic if the planner introduces a particularly sharp turn or, for instance, the robot is unable to come to an immediate stop. Second, our approach assumes the homogeneity of the robots. In reality, not all robots will behave in the same way. Deployments may mix different types of robots. This type of heterogeneity is not supported in PIBT and could be a good future direction for research.

We also could not reduce the makespan of a smoothed path as we relied on the conflicts happening at fixed times dictated by the original plan. A future direction would be to try

to derive a semantic plan instead of a time-based plan and then apply path smoothing in a posthoc fashion. This may enable us to reduce the total makespan.

Additionally, the escape hatch we used to avoid collisions at newly introduced trajectory intersections is less than ideal. Future work can focus on finding a way to ensure collisions do not take place. Alternatively, instead of using SIPP [25], other methods like Optimal Reciprocal Collision Avoidance (ORCA) [22] or Control Barrier Functions [26] (or, for that matter, even PIBT) can be used to resolve such collisions.

Additionally, while this work focused on grid-based safety regions, exploring the robustness of the method in highly dynamic environments with moving obstacles or heterogeneous agents remains an open challenge. Future evaluations could incorporate simple dynamic constraints, such as maximum rotation angles or acceleration limits, to bring the simulated results closer to the requirements of physical robot systems.

Our work also makes no claims about the optimality or completeness of our method. This is an important area for future research. Perhaps we can work such a path smoother into an optimizer like LaCAM* [8] in the future to provide optimality guarantees.

Author Contributions: A.C.: Conceptualization, Software, Formal Analysis, Validation, Writing—Original Draft, M.X.G.: Conceptualization, Validation, Supervision, Writing— Review and Editing, M.A.V.J.M.: Validation, Supervision, Writing—Review and Editing, R.M.E.: Validation, Supervision, Funding Acquisition. All authors have read and agreed to the published version of this manuscript.

Funding: This research was supported by the National Robotics Programme under category National Robotics Programme 2.0, LEO 1.0: A New Class of Bed Making Robot, Award No. M25N4N2028, and by A*STAR under its RIE2025 IAF-PP program, Modular Reconfigurable Mobile Robots (MR)2, grant No. M24N2a0039.

Data Availability Statement: The data presented in this study are openly available fro [GitHub] at [https://github.com/arjo129/pypibt/tree/main/docs/results (accessed on 3 March 2026)].

Conflicts of Interest: Authors Arjo Chakravarty and Michael X. Grey are employed by the company Intrinsic Innovation Singapore Pte Ltd. Arjo Chakravarty is a Ph.D. Student at Singapore University of Technology and Design. The remaining authors declare that this research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Okumura, K.; Machida, M.; Défago, X.; Tamura, Y. Priority Inheritance with Backtracking for Iterative Multi-agent Path Finding. *Artif. Intell.* **2022**, *310*, 103752. [CrossRef]
2. Yakovlev, K.; Andreychuk, A.; Stern, R. Optimal and Bounded Suboptimal Any-Angle Multi-agent Pathfinding. In *Proceedings of the 2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*; IEEE: Piscataway, NJ, USA, 2024; pp. 7996–8001. [CrossRef]
3. Macenski, S.; Martin, F.; White, R.; Ginés Clavero, J. The Marathon 2: A Navigation System. In Proceedings of the 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Las Vegas, NV, USA, 25–29 October 2020.
4. Sharon, G.; Stern, R.; Felner, A.; Sturtevant, N.R. Conflict-based search for optimal multi-agent pathfinding. *Artif. Intell.* **2015**, *219*, 40–66. [CrossRef]
5. Li, J.; Ruml, W.; Koenig, S. Eecbs: A bounded-suboptimal search for multi-agent path finding. In *Proceedings of the AAAI Conference on Artificial Intelligence*; AAAI Press: Palo Alto, CA, USA, 2021; Volume 35, pp. 12353–12362.
6. Shaoul, Y.; Mishani, I.; Vats, S.; Li, J.; Likhachev, M. Multi-robot motion planning with diffusion models. *arXiv* **2024**, arXiv:2410.03072. [CrossRef]
7. Gordon, O.; Filmus, Y.; Salzman, O. Revisiting the Complexity Analysis of Conflict-Based Search: New Computational Techniques and Improved Bounds. In *Proceedings of the Fourteenth International Symposium on Combinatorial Search, SOCS 2021, Online, 16–18 July 2021*; AAAI Press: Palo Alto, CA, USA, 2021; pp. 64–72. [CrossRef]
8. Okumura, K. LaCAM: Search-Based Algorithm for Quick Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*; AAAI Press: Palo Alto, CA, USA, 2023.

9. Surynek, P. Problem Compilation for Multi-Agent Path Finding: A Survey. In *Proceedings of the IJCAI*; Curran Associates, Inc.: Red Hook, NY, USA, 2022; pp. 5615–5622.
10. Surynek, P.; Felner, A.; Stern, R.; Boyarski, E. Sub-optimal sat-based approach to multi-agent path-finding problem. In *Proceedings of the International Symposium on Combinatorial Search*; AAAI Press: Palo Alto, CA, USA, 2018; Volume 9, pp. 99–105.
11. Sartoretti, G.; Kerr, J.; Shi, Y.; Wagner, G.; Kumar, T.S.; Koenig, S.; Choset, H. Primal: Pathfinding via reinforcement and imitation multi-agent learning. *IEEE Robot. Autom. Lett.* **2019**, *4*, 2378–2385. [CrossRef]
12. Wang, Y.; Xiang, B.; Huang, S.; Sartoretti, G. Scrimp: Scalable communication for reinforcement-and imitation-learning-based multi-agent pathfinding. In *Proceedings of the 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*; IEEE: Piscataway, NJ, USA, 2023; pp. 9301–9308.
13. Andreychuk, A.; Yakovlev, K.; Panov, A.; Skrynnik, A. Mapf-gpt: Imitation learning for multi-agent pathfinding at scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*; AAAI Press: Palo Alto, CA, USA, 2025; Volume 39, pp. 23126–23134.
14. Chan, S.H.; Chen, Z.; Guo, T.; Zhang, H.; Zhang, Y.; Harabor, D.; Koenig, S.; Wu, C.; Yu, J. The League of Robot Runners Competition: Goals, Designs, and Implementation. In Proceedings of the ICAPS 2024 System's Demonstration Track, Banff, AB, Canada, 1–6 June 2024.
15. Stern, R.; Sturtevant, N.R.; Felner, A.; Koenig, S.; Ma, H.; Walker, T.T.; Li, J.; Atzmon, D.; Cohen, L.; Kumar, T.K.S.; et al. Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks. *Symp. Comb. Search (SoCS)* **2019**, *10*, 151–158. [CrossRef]
16. GitHub—open-rmf/mapf: Multi-Agent (Path Finding) Planning Framework—github.com. Available online: https://github.com/open-rmf/mapf (accessed on 23 April 2025).
17. Li, J.; Chen, Z.; Harabor, D.; Stuckey, P.J.; Koenig, S. MAPF-LNS2: Fast Repairing for Multi-Agent Path Finding via Large Neighborhood Search. In *Proceedings of the AAAI Conference on Artificial Intelligence*; AAAI Press: Palo Alto, CA, USA, 2022; Volume 36, pp. 10256–10265. [CrossRef]
18. Nash, A.; Daniel, K.; Koenig, S.; Felner, A. Theta*: Any-angle path planning on grids. In *Proceedings of the AAAI*; AAAI Press: Palo Alto, CA, USA, 2007; Volume 7, pp. 1177–1183.
19. Nash, A.; Koenig, S.; Tovey, C. Lazy Theta*: Any-angle path planning and path length analysis in 3D. In *Proceedings of the AAAI Conference on Artificial Intelligence*; AAAI Press: Palo Alto, CA, USA, 2010; Volume 24, pp. 147–154.
20. Cui, X.; Shi, H. Direction oriented pathfinding in video games. *Int. J. Artif. Intell. Appl.* **2011**, *2*, 1. [CrossRef]
21. Van den Berg, J.; Lin, M.; Manocha, D. Reciprocal velocity obstacles for real-time multi-agent navigation. In *Proceedings of the 2008 IEEE International Conference on Robotics and Automation*; IEEE: Piscataway, NJ, USA, 2008; pp. 1928–1935.
22. Alonso-Mora, J.; Breitenmoser, A.; Rufli, M.; Beardsley, P.; Siegwart, R. Optimal reciprocal collision avoidance for multiple non-holonomic robots. In *Proceedings of the Distributed Autonomous Robotic Systems: The 10th International Symposium*; Springer: Berlin/Heidelberg, Germany, 2013; pp. 203–216.
23. Nash, A. Any-Angle Path Planning. Ph.D. Thesis, University of Southern California, Los Angeles, CA, USA, 2012.
24. Surynek, P. An Optimization Variant of Multi-Robot Path Planning Is Intractable. In *Proceedings of the AAAI*; AAAI Press: Palo Alto, CA, USA, 2010; pp. 1261–1263. [CrossRef]
25. Phillips, M.; Likhachev, M. Sipp: Safe interval path planning for dynamic environments. In *Proceedings of the 2011 IEEE International Conference on Robotics and Automation*; IEEE: Piscataway, NJ, USA, 2011; pp. 5628–5635.
26. Gao, Z.; Yang, G.; Prorok, A. Online control barrier functions for decentralized multi-agent navigation. In *Proceedings of the 2023 International Symposium on Multi-Robot and Multi-Agent Systems (MRS)*; IEEE: Piscataway, NJ, USA, 2023; pp. 107–113.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.